

Lebanese University – Faculty of Engineering

Date: 08/07/2026

Subject: Sensors

Professor: Dr. Mohammad Aoude



**Sensors project
(Greenhouse)**

Presented by:

Ahmad Murad - 6856

1. Summary of Contribution

My main responsibility was the firmware architecture running on the Arduino Mega. I structured the main loop into six functions — `checkRFID()`, `readAllSensors()`, `controlOutputs()`, `handleButton()`, `updateLCD()`, and `getDistance()` — and made sure every timed action (fan, buzzer, LEDs) used `millis()`-based non-blocking timers instead of `delay()`, so the system stays responsive to button presses even while sensors are being read.

I implemented the interrupt-driven pulse counting for the YF-S201 flow sensor on D2, including the logic to briefly detach the interrupt during the 1-second calculation window so pulse counts don't get corrupted mid-read. I also wrote the threshold-based automation logic: the irrigation pump turning on when soil ADC exceeds 500, the fan activating above 29°C, the grow-LEDs triggering when the LDR reading drops below 10, and the buzzer/sound alarm timing windows.

2. Git Activity

- Scaffold main loop with six core functions
- Implement non-blocking timers for fan and buzzer
- Add interrupt handler for YF-S201 flow sensor on D2
- Add soil-moisture threshold logic for pump control
- Add temperature threshold logic for fan control
- Detach/reattach flow interrupt during calc window
- Tune buzzer and sound-alarm timing windows

3. Reflection

This was my first time writing a non-trivial embedded control loop without using `delay()`, and it changed how I think about timing in general — everything became a matter of tracking `millis()` differences instead of blocking the whole program. The trickiest part was getting the flow-sensor interrupt to behave correctly without dropping pulses; understanding why detaching it briefly mattered gave me a much better feel for how interrupts interact with the rest of a program.